Reinforcement Learning

Multi-agent Systems & Agentic AI

Lecture Notes

Easter Term 2026

Marta Grzeskiewicz
mg2192@cam.ac.uk

Course Map

Lectures

1. **Foundations of RL**
   MDP, policies, value functions
2. **Policy Gradient Methods**
   REINFORCE, baselines, advantage
3. **GRPO**
   Group Relative Policy Optimization
4. **Theory**
   Variance, bias, convergence
5. **Agentic Alignment**
   RLHF as system, safety, failures

Running Thread

Each lecture builds toward a single question:

How do we train an LLM to behave as intended?

Lectures 1–2: classical tools
Lecture 3: GRPO + RLHF bridge
Lecture 4: why it (sometimes) works
Lecture 5: alignment as a system

Lecture 1

Foundations of Reinforcement Learning

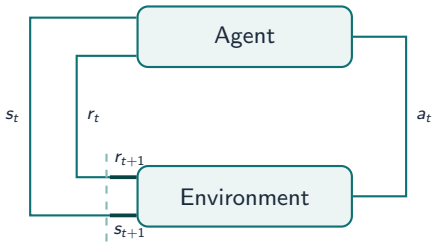
Why Reinforcement Learning?

Supervised learning needs labelled targets.

Many desirable behaviours are **easier to evaluate than to demonstrate**:

- Winning a chess position
- Writing a helpful answer
- Following a user's instruction safely

RL: learn from **scalar feedback** (reward) obtained by acting.



Example: AlphaGo

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

Symbol	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$P(s' \mid s, a)$	transition kernel
$R(s, a)$	expected reward
$\gamma \in [0, 1)$	discount factor

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

Symbol	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$P(s' \mid s, a)$	transition kernel
$R(s, a)$	expected reward
$\gamma \in [0, 1)$	discount factor

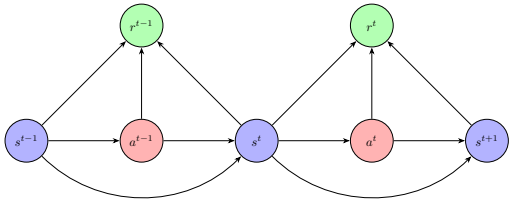


Figure 1: MDP Transition Loop

The Markov Decision Process (MDP)

Definition

An MDP is defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

Symbol	Meaning
$\mathcal{S}$	state space
$\mathcal{A}$	action space
$P(s' \mid s, a)$	transition kernel
$R(s, a)$	expected reward
$\gamma \in [0, 1)$	discount factor

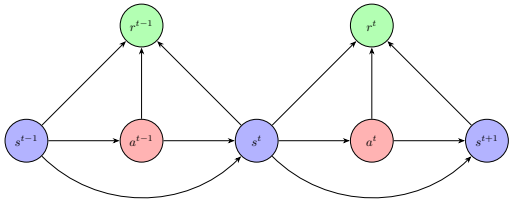


Figure 1: MDP Transition Loop

Markov property

$P(s_{t+1} \mid s_t, a_t, s_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$

The future is conditionally independent of the past given the present state.

Policies

Definition

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

Policies

Definition

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

Stochastic policy

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

Policies

Definition

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

Stochastic policy

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

Deterministic policy

$$\mu : \mathcal{S} \to \mathcal{A}$$

- Special case: $\pi(a|s) = \mathbf{1}[a = \mu(s)]$
- Useful for continuous control
- Not directly applicable to LLMs

Policies

Definition

A **policy** $\pi : \mathcal{S} \to \Delta(\mathcal{A})$ maps states to distributions over actions.

Stochastic policy

$$\pi(a \mid s) = P(A_t = a \mid S_t = s)$$

- Enables exploration
- Required for policy gradient theory
- Standard in LLM fine-tuning

Deterministic policy

$$\mu : \mathcal{S} \to \mathcal{A}$$

- Special case: $\pi(a|s) = \mathbf{1}[a = \mu(s)]$
- Useful for continuous control
- Not directly applicable to LLMs

Later in this course: π_θ is a language model parameterised by weights θ.

The RL Objective

Define the **return** (total discounted reward) from timestep t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The RL Objective

Define the **return** (total discounted reward) from timestep t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **RL objective** is to maximise:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right]$$

The RL Objective

Define the **return** (total discounted reward) from timestep t:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

The **RL objective** is to maximise:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right]$$

- $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots)$ is a **trajectory**
- $\gamma < 1$ ensures convergence for infinite horizons
- Choice of γ encodes *time preference*

Trajectories and Their Distribution

Trajectory distribution

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} \mid s_t, a_t)\, \pi_\theta(a_t \mid s_t)$$

- ρ_0: initial state distribution
- The policy influences which trajectories are visited
- **Key insight**: gradient of $J(\theta)$ must account for the changing distribution

Value Functions

State-value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state s following π.

Value Functions

State-value function

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Expected return starting from state s following π.

Action-value function

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a]$$

Expected return taking action a from s, then following π.

Value Functions

State-value function

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

Expected return starting from state s following π.

Action-value function

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

Expected return taking action a from s, then following π.

Advantage function

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

How much better is action a than the *average* action in state s?

State-Value Functions

State-value functions $V^{\pi}(s)$ give the 'value' of state s when following the policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t | S_t = s\right]$$

State-Value Functions

State-value functions $V^{\pi}(s)$ give the 'value' of state s when following the policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t | S_t = s\right]$$

The return (u) can be recursively defined as:

$$\begin{aligned} G_t &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + ...) \\ &= r_t + \gamma G_{t+1} \end{aligned}$$

State-Value Functions

State-value functions $V^{\pi}(s)$ give the 'value' of state s when following the policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t | S_t = s\right]$$

The return (u) can be recursively defined as:

$$\begin{aligned} G_t &= r_t + \gamma(r_{t+1} + \gamma r_{t+2} + ...) \\ &= r_t + \gamma G_{t+1} \end{aligned}$$

Therefore:

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[r_t + \gamma G_{t+1} | S_t = s\right]$$

The Bellman Equation

$$V^{\pi}(s) = \mathbb{E}_{\pi}[r_t + \gamma G_{t+1} \mid S_t = s] \tag{1}$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s_t) \sum_{s' \in \mathcal{S}} P(s'|s,a) \left[R(s,a) + \gamma \mathbb{E}_{\pi}[G_{t+1} \mid S_{t+1} = s']\right] \tag{2}$$

$$= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{s' \in \mathcal{S}} P(s' \mid s,a) \left[R(s,a) + \gamma V^{\pi}(s')\right] \tag{3}$$

The last equation is known as the **Bellman equation** in honour of Richard Bellman.

State-Action Value Function

State-action value functions $Q^{\pi}(s, a)$ are an extension on the *State* value functions. They condition the expected return on the current state and the action taken:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[G_t | S_t = s, A_t = a\right]$$

The *state-action* value Bellman equation is therefore:

$$Q^{\pi}(s, a) = \sum_{s' \in S} P(s'|s, a)\left[\mathcal{R}(s, a) + \gamma V^{\pi}(s')\right]$$

Why the Advantage Function Matters

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

Why the Advantage Function Matters

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- $A^{\pi}(s,a) > 0 \Rightarrow$ action a is **better than average**; increase its probability

Why the Advantage Function Matters

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

- $A^\pi(s,a) > 0 \Rightarrow$ action a is **better than average**; increase its probability
- $A^\pi(s,a) < 0 \Rightarrow$ action a is **worse than average**; decrease its probability

Why the Advantage Function Matters

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

- $A^\pi(s,a) > 0 \Rightarrow$ action a is **better than average**; increase its probability
- $A^\pi(s,a) < 0 \Rightarrow$ action a is **worse than average**; decrease its probability
- $A^\pi(s,a) = 0 \Rightarrow$ indifferent; no update needed

Why the Advantage Function Matters

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

- $A^\pi(s,a) > 0 \Rightarrow$ action a is **better than average**; increase its probability
- $A^\pi(s,a) < 0 \Rightarrow$ action a is **worse than average**; decrease its probability
- $A^\pi(s,a) = 0 \Rightarrow$ indifferent; no update needed

Baseline intuition

Subtracting $V^\pi(s)$ from $Q^\pi(s,a)$ gives a *centered* signal.
This is the single most important variance-reduction idea in RL.

RL as Optimisation over Distributions

Key Reframing

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \int p_{\pi_\theta}(\tau)\left(\sum_{t=0}^{T} \gamma^t r_t\right) d\tau$$

- θ shapes the *distribution* $p_\theta(\tau)$; we optimise over distributions
- This is the fundamental challenge: the expectation depends on θ
- Standard autodiff cannot directly differentiate through *sampling*
- Solution: the **log-derivative trick** (Lecture 2)

Why Not Value Iteration?

Classical **value-based methods** (Q-learning, value iteration):

- Enumerate $\mathcal{S} \times \mathcal{A}$ — infeasible for LLMs ($|\mathcal{A}| \approx 50{,}000$ tokens, sequences of length $\sim 10^3$)
- Greedy extraction breaks differentiability
- Poor generalisation across states

LLM Scale

A single output sequence is an action. The action space is combinatorially huge.

Direct policy optimisation with gradient descent is the *only* scalable approach.

Lecture 1 — Summary

Core take-aways

1. RL formalises **sequential decision-making** via MDPs
2. A **policy** $\pi(a|s)$ is what we learn; the **objective** is to maximise the expected discounted return:
$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right]$$
3. **Value functions** V^π, Q^π, A^π measure long-run quality
4. The **advantage function** is the key quantity for policy updates
5. RL = optimise a policy over a *distribution of trajectories*

Next: how do we compute $\nabla_\theta J(\theta)$?

Lecture 2

Policy Gradient Methods

The Policy Gradient Objective

We want $\nabla_\theta J(\theta)$ where:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

Challenge: $p_\theta(\tau)$ depends on θ, so we cannot simply push ∇_θ inside the integral naively.

The Policy Gradient Objective

We want $\nabla_\theta J(\theta)$ where:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T} \gamma^t r_t \right] = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$$

Challenge: $p_\theta(\tau)$ depends on θ, so we cannot simply push ∇_θ inside the integral naively.

Log-derivative (REINFORCE) trick

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$$

because $\nabla_\theta \log p = \frac{\nabla_\theta p}{p}$.

The Log-Derivative Trick

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int \nabla_\theta p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot G(\tau)]
\end{aligned}
$$

The Log-Derivative Trick

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int \nabla_\theta p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, G(\tau)\, d\tau \\
&= \mathbb{E}_{\tau \sim p_\theta}[\nabla_\theta \log p_\theta(\tau) \cdot G(\tau)]
\end{aligned}
$$

Since $\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_t \log P(s_{t+1}|s_t, a_t) + \sum_t \log \pi_\theta(a_t|s_t)$, and only the last term depends on θ:

$$
\boxed{\nabla_\theta J(\theta) \propto \mathbb{E}_{\tau \sim p_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G(\tau)\right]}
$$

The REINFORCE Algorithm

REINFORCE (Williams, 1992)

1. Sample trajectory $\tau \sim \pi_\theta$
2. Compute return $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ for each step t
3. Update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

The REINFORCE Algorithm

REINFORCE (Williams, 1992)

1. Sample trajectory $\tau \sim \pi_\theta$
2. Compute return $G_t = \sum_{k \geq t} \gamma^{k-t} r_k$ for each step t
3. Update:

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t$$

- Unbiased gradient estimate
- No model of P, no value function required

Exploration vs. Exploitation

The core tension

Exploit: repeat actions already known to give high reward.

Explore: try new actions that might be better — but carry risk.

LLM connection

Temperature T controls this trade-off at inference time.

High $T \rightarrow$ flat $\pi_\theta \rightarrow$ exploration.
Low $T \rightarrow$ sharp $\pi_\theta \rightarrow$ exploitation.

How REINFORCE handles it implicitly

- A *stochastic* policy $\pi_\theta(a|s)$ explores by construction — it never deterministically commits to one action.
- As training progresses, high-reward actions accumulate probability mass: the policy becomes increasingly deterministic.
- Left unchecked, this causes **premature convergence**: the agent stops exploring before reaching the global optimum.

Code: REINFORCE — Policy and Episode

```python
import torch, torch.nn as nn

class Policy(nn.Module):
    def __init__(self, n_obs, n_act):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_obs, 64),
            nn.ReLU(),
            nn.Linear(64, n_act),
        )

    def forward(self, obs):
        logits = self.net(obs)
        # stochastic policy = Categorical
        return torch.distributions.Categorical(
            logits=logits
        )

def collect_episode(env, policy):
    obs = env.reset()
    log_probs, rewards = [], []
    done = False
    while not done:
        dist = policy(torch.tensor(obs, dtype=torch.float))
        action = dist.sample()
        log_probs.append(dist.log_prob(action))
        obs, r, done, _ = env.step(action.item())
        rewards.append(r)
    return log_probs, rewards
```

Key lines:

- `Categorical(logits=)` is $\pi_\theta(a|s)$
- `dist.sample()` — stochastic action
- `dist.log_prob(a)` — the score function $\nabla_\theta \log \pi$ will come from autodiff on this
- No value network, no critic

Code: REINFORCE — Update Step

```python
def reinforce_update(log_probs, rewards,
                     optimizer, gamma=0.99):
    # 1. Discounted returns
    G, returns = 0.0, []
    for r in reversed(rewards):
        G = r + gamma * G
        returns.insert(0, G)
    returns = torch.tensor(returns, dtype=torch.float)

    # 2. Standardise (baseline = mean return)
    returns = (returns - returns.mean()) \
              / (returns.std() + 1e-8)

    # 3. Policy gradient loss
    log_probs_t = torch.stack(log_probs)
    loss = -(log_probs_t * returns).sum()

    # 4. SGD step
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    return loss.item()
```

What to watch when you run this:

- `loss` fluctuates wildly — this *is* the variance problem
- Gradient norms vary by $10\times$ across episodes
- Convergence is slow: hundreds of episodes for CartPole

Reference

Spinning Up:
`spinningup.openai.com`
Clean minimal PyTorch implementations of REINFORCE $\rightarrow$ PPO.

The Variance Problem

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}$$

Sources of variance:

- Returns G_t accumulate noise over the whole trajectory
- Stochastic transitions multiply uncertainty
- Long horizons: variance scales as $O(T^2)$

Consequence

In practice REINFORCE needs thousands of samples and struggles.

Baselines for Variance Reduction

Baseline theorem

For any function $b(s)$ that does **not** depend on a:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \big(G_t - b(s_t)\big)\right]$$

The estimate remains **unbiased**.

Why? $\mathbb{E}_\pi[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)] = 0$ because:

$$\sum_a \pi_\theta(a|s)\, \nabla_\theta \log \pi_\theta(a|s) = \nabla_\theta \sum_a \pi_\theta(a|s) = \nabla_\theta 1 = 0$$

Proof: Baseline has Zero Expectation

$$\begin{aligned}
\mathbb{E}_\pi\Big[\nabla_\theta \log \pi_\theta(a|s) \cdot b(s)\Big] &= \int \pi_\theta(a|s)\, \nabla_\theta \log \pi_\theta(a|s) \cdot b(s)\; da \\
&= \int \nabla_\theta \pi_\theta(a|s) \cdot b(s)\; da \\
&= b(s)\, \nabla_\theta \underbrace{\int \pi_\theta(a|s)\; da}_{=1} = 0
\end{aligned}$$

A baseline subtracts a constant from the reward signal without changing the gradient direction.

Common choice: $b(s_t) = V^\pi(s_t)$, giving the **advantage** $G_t - V^\pi(s_t) \approx A^\pi(s_t, a_t)$.

Policy Gradient with Advantage

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A^\pi(s_t, a_t)\right]$$

Advantage Actor-Critic (A2C)

- **Actor**: policy π_θ — what to do
- **Critic**: value estimator $\hat{V}_\phi(s)$ — how good is the current state
- Advantage: $\hat{A}(s_t, a_t) = r_t + \gamma \hat{V}_\phi(s_{t+1}) - \hat{V}_\phi(s_t)$

Problem in LLMs

Learning a reliable critic $\hat{V}_\phi$ for natural-language states is *hard*.
GRPO (Lecture 3) eliminates the critic entirely.

Monte Carlo vs. Bootstrapping

Monte Carlo

$G_t = \sum_{k \geq t} \gamma^{k-t} r_k$

- Unbiased
- High variance
- Requires complete episodes

Temporal Difference (TD)

$G_t \approx r_t + \gamma \hat{V}(s_{t+1})$

- Biased (uses estimate)
- Lower variance
- Works online

GAE (Generalised Advantage Estimation) interpolates via $\lambda \in [0, 1]$:

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{k=0}^{\infty} (\gamma\lambda)^k \delta_{t+k}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Credit Assignment Noise

Problem

Which actions caused the high return? In a long trajectory, early and late rewards are entangled.

Causal structure: reward at t should only influence updates for actions $a_{t' \leq t}$:

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot \underbrace{\sum_{k \geq t} \gamma^{k-t} r_k}_{G_t}\right]$$

REINFORCE already captures this via G_t (future rewards only); the baseline sharpens it further.

Why this rules out a replay buffer

A replay buffer stores transitions (s_t, a_t, r_t, s_{t+1}) collected under an *old* policy $\pi_{\theta_{\text{old}}}$.

When we replay them under $\pi_\theta \neq \pi_{\theta_{\text{old}}}$, the credit assignment is doubly broken:

- G_t was computed under $\pi_{\theta_{\text{old}}}$ — the return reflects the old policy's downstream behaviour, not what π_θ would have done after a_t.
- The score $\nabla_\theta \log \pi_\theta(a_t|s_t)$ now pushes θ based on a reward signal that was never causally linked to π_θ.

Upshot: policy gradient must be on-policy, or use importance weighting — the cost GRPO avoids by treating each prompt as a single-step reward.

Towards Stable Policy Updates: PPO

The problem with large gradient steps

A big update to θ can move π_θ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

Policy collapse: a few bad steps can destroy a well-trained policy irreversibly.

Towards Stable Policy Updates: PPO

The problem with large gradient steps

A big update to θ can move π_θ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

Policy collapse: a few bad steps can destroy a well-trained policy irreversibly.

TRPO (2015) formalises this as a KL-constrained optimisation problem.

Towards Stable Policy Updates: PPO

The problem with large gradient steps

A big update to θ can move π_θ far from the policy that collected the data. The gradient estimate is now stale — and the next update is worse, not better.

Policy collapse: a few bad steps can destroy a well-trained policy irreversibly.

TRPO (2015) formalises this as a KL-constrained optimisation problem.

Proximal Policy Optimisation (PPO) (Schulman et al., 2017)

Replace the constraint with a **clipped surrogate objective**:

$$J^{\mathrm{PPO}}(\theta) = \mathbb{E}_t\big[\min\big(r_t A_t,\ \mathrm{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\, A_t\big)\big]$$

where $r_t(\theta) = \pi_\theta(a_t|s_t) \,/\, \pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ is the **importance ratio** — how much the policy has moved.

The clip removes the incentive to push r_t outside $[1-\varepsilon,\ 1+\varepsilon]$, bounding each step.

Lecture 2 — Summary

Core take-aways

1. **Log-derivative trick**: $\nabla_\theta J = \mathbb{E}\big[\nabla_\theta \log \pi_\theta \cdot G\big]$
2. **REINFORCE**: unbiased, but high-variance gradient estimator
3. **Baselines** reduce variance without introducing bias
4. **Advantage** $A^\pi(s, a)$ is the centred signal of choice
5. **Actor-Critic**: use a learned V^π as baseline — but critic training is hard

Next: eliminate the critic entirely using group comparisons.